



Republic of Tunisia
*** * ***
Ministry of Youth and Sports
*** * ***
Discovery
Club Junior



ENGINEERING JOURNAL

DISCOVERY CLUB JUNIOR

ROBOCUP GERMANY 2024

Topic:

RCJ Rescue Maze Robot

"The Maze Runner"

"HATREF"

Prepared by:

Maram Ouelhazi
Mohammed Rayen Hammoudi
Anwer Kadhraoui
Mohammed Omar Ferchichi

Year: 2023/2024

Contents

1	Presentation	2
1.1	Introduction	2
1.2	Problematic	2
2	Conception	3
2.1	Competition Requirements	3
2.2	Design and Development Process	3
2.2.1	Conceptual Design	4
2.2.2	Mechanical Design	4
2.3	Hardware Components	6
2.3.1	Microcontroller	6
2.3.2	Motors and Encoders	6
2.3.3	Sensors	7
2.3.4	Color Sensor	7
2.3.5	Line Follower Sensor	8
2.3.6	Limit Switch	9
2.3.7	Image Processing Unit	9
3	Software Architecture (Arduino)	11
3.1	Initialization and Setup	11
3.2	Sensor Integration	11
3.3	Navigation and Mapping	12
3.4	Algorithm Choice	12
3.5	Data Structures	13
3.6	Maze Representation	13
3.7	Heuristics	14
3.8	Path Construction	14
3.9	Edge Cases	15
3.10	Performance	15
3.11	Conclusion	15
3.12	Movement Functions	16
3.13	Control Algorithms	17
4	Software Architecture (Raspberry)	18
4.1	Initialization and Setup	18
4.1.1	Importing essential libraries:	18
4.1.2	Configuring the serial port:	18
4.2	Sensor Integration	19

4.3	Navigation and Mapping	19
4.4	Algorithm Choice	19
4.4.1	Implementing color detection algorithms:	19
4.5	Data Structures	20
4.5.1	Using Numpy arrays:	20
4.6	Maze Representation	20
4.6.1	Isolating colors using masks:	20
4.7	Heuristics	21
4.7.1	Using Tesseract OCR for text detection:	21
4.8	Path Construction	21
4.8.1	Decision-making based on detections:	21
4.9	Edge Cases	22
4.10	Performance	22
4.11	Conclusion	22
5	Control and Navigation Algorithms	23
5.1	PID Control	23
5.1.1	Proportional Term (P)	23
5.1.2	Integral Term (I)	23
5.1.3	Derivative Term (D)	24
5.1.4	Combined PID Equation	24
5.2	Angle Calculations	26
5.3	Map Navigation	26
5.3.1	Distance Measurement	26
5.3.2	Direction Decision	27
5.3.3	Movement Execution	27
6	Performance and Testing(Arduino)	28
6.1	Simulation Testing	28
6.1.1	Setup and Environment	28
6.1.2	Scenarios and Testing Hours	28
6.1.3	Performance Metrics	29
6.2	Real-World Testing	29
6.2.1	Setup and Environment	29
6.2.2	Scenarios and Testing Hours	29
6.3	Detailed Observations	30
6.3.1	Accuracy of Distance Sensors	30
6.3.2	Gyroscope Performance	30
6.3.3	PID Control Efficiency	30
7	Performance and Testing (Raspberry Pi)	31
7.0.1	Performance Analysis	31
7.0.2	Testing Procedure	33
7.0.3	Creating a Service to Run on Raspberry Pi 5 Boot	33
7.1	Conclusion	34
8	Conclusion	35
9	Future Improvements	36

List of Figures

2.1	Different Views of The Robot	5
2.2	Arduino Card Mega 2560	6
2.3	Motor 12V DC with Encoder 157rpm	6
2.4	IR Distance Sensor GP2Y0A41SK0F (Sharp, Analog, 4-30 cm)	7
2.5	MPU6050 Gyroscope/Accelerometer Sensor	7
2.6	HiLetgo TCS-34725 TCS34725 RGB Light Color Sensor Colour Recognition Module	8
2.7	Line follower sensor - TCRT500 module, analog and digital output	8
2.8	Limit Switch Module V-156-1C25	9
2.9	Raspberry Pi 5 (4GB)	9
2.10	Redragon Hitman GW800 1080P FHD Webcam	10
3.1	Flow Chart for DFS Algorithm	12
3.2	Depth First Traversal DFS on a 2D array	14
3.3	Maze Path-finding using DFS	16
5.1	PID's Diagrams	25
7.1	Detection of the letter U	31
7.2	Detection of the letter H	32
7.3	Detection of the colour green	32
7.4	Detection of the colour red	32

Chapter 1

Presentation

1.1 Introduction

The RCJ Rescue Maze Robot represents a sophisticated fusion of engineering, software architecture, and sensor integration tailored to address the intricate challenges of maze navigation. In a landscape where robotics continually reshapes industry standards, our endeavor with the RCJ Rescue Maze Robot exemplifies a dedication to precision engineering and innovative problem-solving.

1.2 Problematic

In the maze, the RCJ Rescue Maze Robot faces numerous obstacles and challenges. Success requires blending real-time data processing, adaptive algorithms, and seamless sensor integration. Balancing efficiency and accuracy, the robot must swiftly detect and navigate around obstacles. Our goal is to engineer a platform that excels in precision, adaptability, and reliability, pushing the boundaries of autonomous maze navigation technology.

Navigating the maze demands intricate sensor integration and rapid obstacle detection. We aim to engineer a robot that excels in precision and adaptability, setting new standards in autonomous navigation technology. Our commitment to technical excellence drives us to push the boundaries of robotic capabilities in maze environments.

Chapter 2

Conception

2.1 Competition Requirements

The competition has outlined several key requirements and constraints that guided the design of the robot:

1. **Autonomous Navigation:** The robot must navigate the maze autonomously, without any external control or intervention.
2. **Obstacle Detection:** The robot must detect and avoid obstacles within the maze.
3. **Letter and Color Recognition:** The robot must identify letters on the walls and distinguish between different colors.
4. **Precision in Movement:** The robot must exhibit precise movement control to navigate through narrow paths and make accurate turns.
5. **Stability and Power Management:** The robot must be stable and have an efficient power management system to operate for extended periods.

2.2 Design and Development Process

The design of the RCJ Rescue Maze Robot was an iterative process involving several key stages:

2.2.1 Conceptual Design

Initially, the team brainstormed various designs that would meet the competition requirements. The RCJ Rescue Maze Robot is built upon a robust and modular structural framework. The following considerations were made:

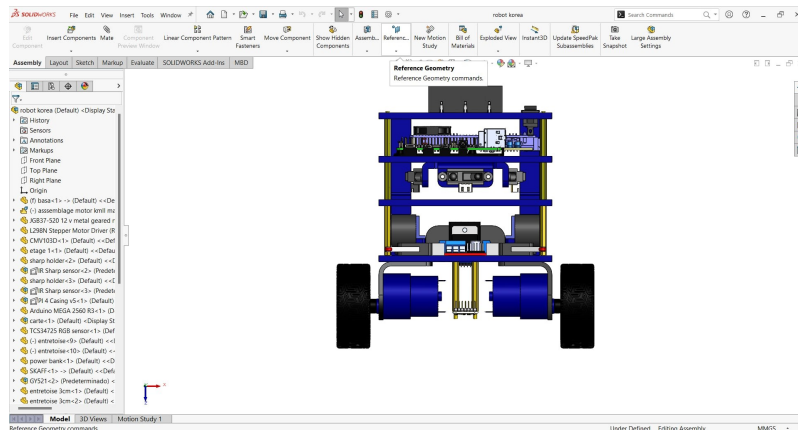
- **Modular Structure:** The robot should have a modular design to facilitate easy assembly and maintenance.
- **Sensor Placement:** Optimal placement of sensors to ensure accurate detection of obstacles and letters/colors.
- **Power Management:** Efficient power distribution and management system to ensure all components function reliably.

2.2.2 Mechanical Design

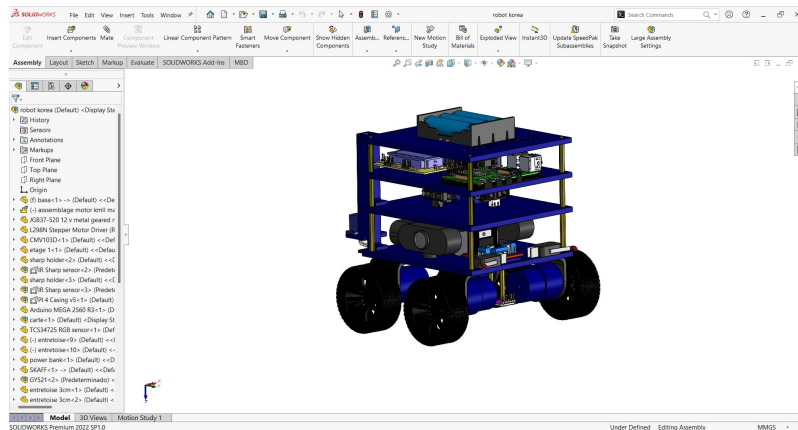
The robot's mechanical design was developed using CAD software, ensuring precise dimensions and placements for all components. The final design featured a multi-tiered structure, comprising:

- **Bottom Tier:**
 - **Components:** Arduino microcontroller, motor driver modules, and other electronic cards, line follower, color sensor, 2 limit switch.
 - **Purpose:** Central processing and control unit for motor and sensor integration.
- **Middle Tier:**
 - **Components:** Four Sharp infrared distance sensors.
 - **Placement:** Each sensor facing a different side (front, back, left, right) for comprehensive obstacle detection.
- **Upper Tier:**
 - **Components:** Camera modules and Raspberry Pi 5.
 - **Purpose:** Image processing for letter and color recognition.
- **Top Tier:**

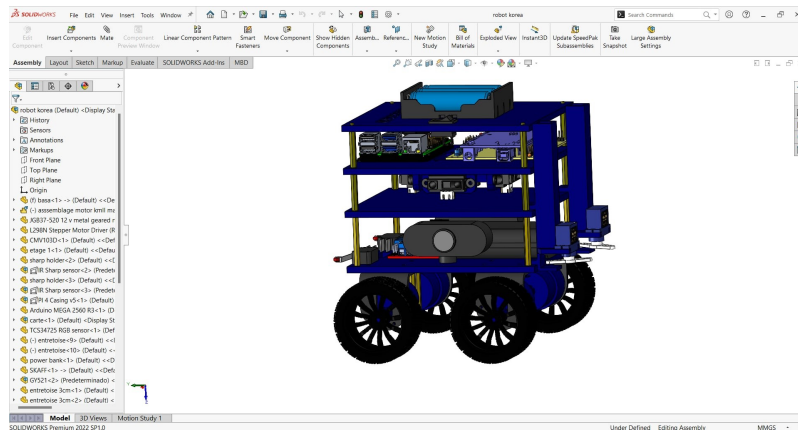
- **Components:** Power bank and lithium batteries.
- **Purpose:** Power supply for all components, ensuring stable and long-term operation.



Side View of RCJ Rescue Maze Robot



Three-Quarter View of RCJ Rescue Maze Robot



Front View of RCJ Rescue Maze Robot

Figure 2.1: Different Views of The Robot

2.3 Hardware Components

The robot consists of the following hardware components:

2.3.1 Microcontroller

- **Arduino:** Used as the central processing unit.
- Chosen for its simplicity, ease of programming, and wide community support. It provides sufficient I/O pins and processing power for the robot's tasks.

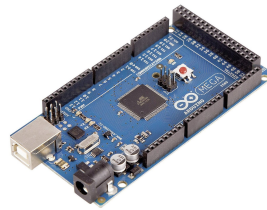


Figure 2.2: Arduino Card Mega 2560

2.3.2 Motors and Encoders

- **Two DC motors with quadrature encoders:** For precise movement control.
- Provide precise control over the robot's movements, essential for accurate navigation in the maze.



Figure 2.3: Motor 12V DC with Encoder 157rpm

2.3.3 Sensors

- **Infrared Distance Sensors:** Four Sharp infrared sensors (SharpR, SharpL, SharpF, SharpB) for obstacle detection.
- Selected for their accuracy in distance measurement and robustness in different lighting conditions.

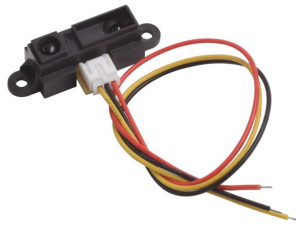


Figure 2.4: IR Distance Sensor GP2Y0A41SK0F (Sharp, Analog, 4-30 cm)

- **MPU6050:** Gyroscope and accelerometer for orientation and movement tracking.
- Offers reliable orientation data, critical for maintaining the robot's direction and calculating turns.



Figure 2.5: MPU6050 Gyroscope/Accelerometer Sensor

2.3.4 Color Sensor

- **TCS34725:** Color sensor for color recognition.
- Enables the robot to accurately distinguish between different colors, which is essential for recognizing certain maze tiles.

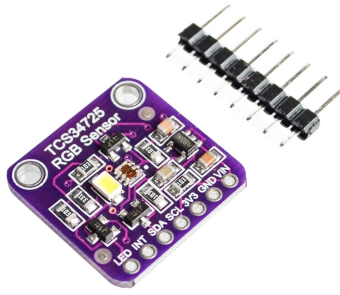


Figure 2.6: HiLetgo TCS-34725 TCS34725 RGB Light Color Sensor Colour Recognition Module

2.3.5 Line Follower Sensor

- **TCRT500:** Infrared line follower sensor for tracking lines on the maze floor.
- Allows the robot to recognize black maze tiles.

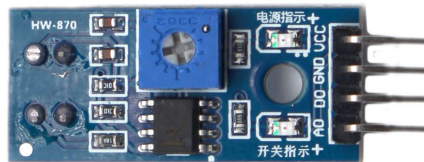


Figure 2.7: Line follower sensor - TCRT500 module, analog and digital output

2.3.6 Limit Switch

- **V-156-1C25**: Limit switch for detecting physical limits or boundaries.
- Useful for ensuring the robot stops or changes behavior when reaching the end of a track or encountering specific barriers.



Figure 2.8: Limit Switch Module V-156-1C25

2.3.7 Image Processing Unit

- **Raspberry Pi 5**: Used for image processing tasks, including recognizing letters on walls and detecting colors.

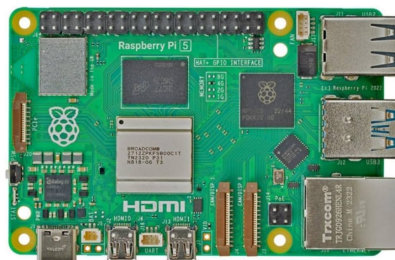


Figure 2.9: Raspberry Pi 5 (4GB)

- **Two Camera Modules:** Attached to the Raspberry Pi for capturing images.



Figure 2.10: Redragon Hitman GW800 1080P FHD Webcam

Chapter 3

Software Architecture (Arduino)

The software is structured in the following key modules:

3.1 Initialization and Setup

- Configures pin modes for sensors and motors.
- Initializes the MPU6050 sensor and calculates its offsets.
- Sets up the timer interrupt for periodic updates.
- Initializes the maze matrix and adds the starting coordinates to the navigation pile.

3.2 Sensor Integration

- **Infrared Distance Sensors:** Used to measure distances to obstacles. The `distance(int pin)` function reads analog values, converts them to voltage, and calculates the distance using a pre-determined formula.
- **Gyroscope:** Integrated via the MPU6050 to track the robot's orientation. The orientation data is used to make precise turns and maintain direction.

3.3 Navigation and Mapping

3.4 Algorithm Choice

The Depth-First Search (DFS) algorithm was selected for solving the maze due to its simplicity and effectiveness in certain types of mazes. DFS is a fundamental graph traversal algorithm that explores as far as possible along each branch before backtracking. This characteristic makes it well-suited for exploring mazes with deep paths and fewer dead ends. The key reasons for choosing DFS include:

- **Simple Implementation:** DFS is straightforward to implement using basic data structures such as stacks and sets.
- **Memory Efficiency:** DFS requires less memory compared to algorithms like Breadth-First Search (BFS) since it stores only a single path from the root to the leaf node.
- **Optimal for Certain Mazes:** In mazes where the solution is located deep within the search tree, DFS can find the solution faster than other algorithms.

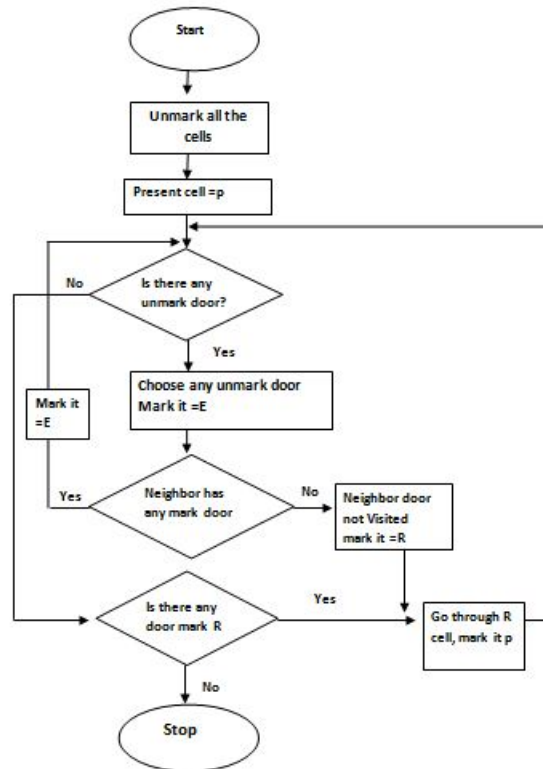


Figure 3.1: Flow Chart for DFS Algorithm

3.5 Data Structures

The DFS algorithm utilizes two primary data structures:

- **Stack for Path Exploration:** A stack is used to keep track of the current path and explore nodes in a last-in, first-out (LIFO) manner. This allows the algorithm to backtrack efficiently when a dead end is reached.
- **Set for Visited Nodes:** A set is used to store visited nodes, ensuring that each node is processed only once. This prevents the algorithm from revisiting nodes and getting stuck in loops.

3.6 Maze Representation

The maze is represented as a 70x70 grid, where each cell contains a 5-bit value indicating the presence of walls and whether the cell has been visited. The structure and navigation logic of the maze are implemented as follows:

```
struct FiveBitValue {
    byte value : 5;
};

const int matrixSize = 70;
FiveBitValue maze[matrixSize][matrixSize];

void initializeMatrix() {
    for (int i = 0; i < matrixSize; i++) {
        for (int j = 0; j < matrixSize; j++) {
            maze[i][j].value = 0b0;
        }
    }
}
```

3.7 Heuristics

While DFS does not utilize heuristics, it's essential to understand its differences compared to heuristic-based algorithms like A*. A* uses a heuristic function to estimate the cost from the current node to the goal, prioritizing paths that seem more promising. DFS, in contrast, explores all possible paths to their maximum depth before backtracking, which can be less efficient in mazes with many branches or loops.

3.8 Path Construction

The path is constructed by pushing nodes onto the stack as they are explored. When the goal is reached, the path can be traced back by following the nodes in the stack. If a dead end is encountered, the algorithm backtracks by popping nodes from the stack until a new unexplored path is found. The following functions illustrate the movement and path construction:

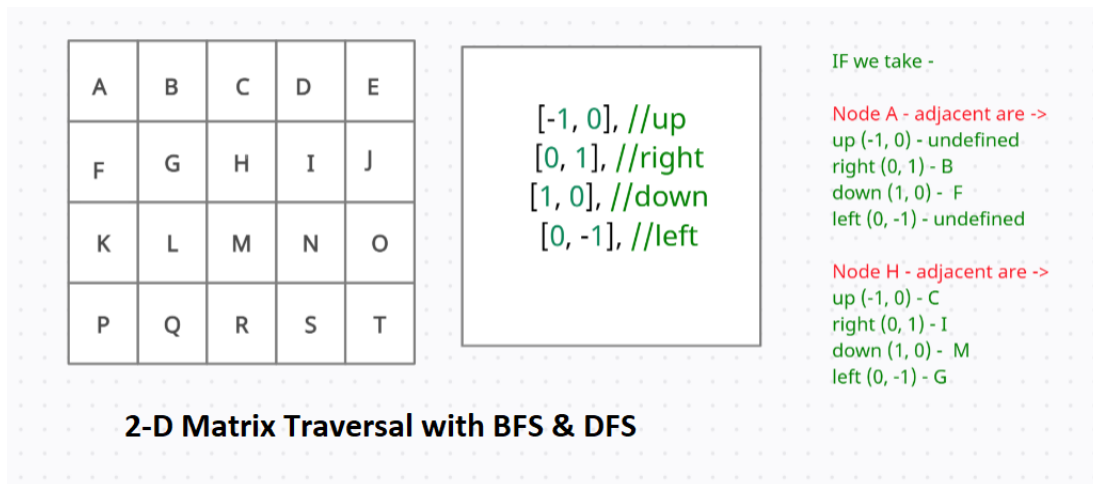


Figure 3.2: Depth First Traversal DFS on a 2D array

```
void move(float distance, float speed) {  
    iniiit();  
    float accel = 0.25 * distance;  
    float decel = 0.75 * distance;  
  
    while (abs(dC_total - distance) > 5) {
```

```

    // Path construction logic
    // Control algorithms for movement
    run();
}
STOP();
}

```

3.9 Edge Cases

Edge cases considered include:

- **No Possible Path:** If no path to the goal exists, the algorithm will eventually exhaust all possible paths and return failure.
- **Handling Large Mazes:** For large mazes, the stack can grow significantly, potentially leading to memory constraints. Efficient memory management and pruning strategies are essential to handle large mazes effectively.

3.10 Performance

The performance of the DFS algorithm is characterized by its time and space complexity:

- **Time Complexity:** The time complexity of DFS is $O(V + E)$, where V is the number of vertices (nodes) and E is the number of edges. This ensures that every node and edge is explored once.
- **Space Complexity:** The space complexity is $O(V)$ due to the storage requirements of the stack and visited set.

3.11 Conclusion

The Depth-First Search algorithm is a robust choice for maze solving in the RCJ Rescue Maze Robot. Its simplicity, memory efficiency, and suitability for certain types of mazes make it a practical solution. By carefully managing the stack and visited nodes, the algorithm ensures efficient path exploration and construction. While it lacks heuristics, its performance is adequate

for the maze configurations encountered in the competition. Future improvements could involve integrating heuristic-based algorithms for enhanced efficiency in more complex maze scenarios.

- **Maze Representation:** The maze is represented as a 70x70 matrix, where each cell is a 5-bit value indicating walls and visited status.
- **Pathfinding Algorithm:** The robot uses a simple stack-based approach to navigate. It checks for possible movements (up, down, left, right) based on sensor data and updates its position accordingly. The algorithm ensures that the robot does not revisit already explored cells unnecessarily.

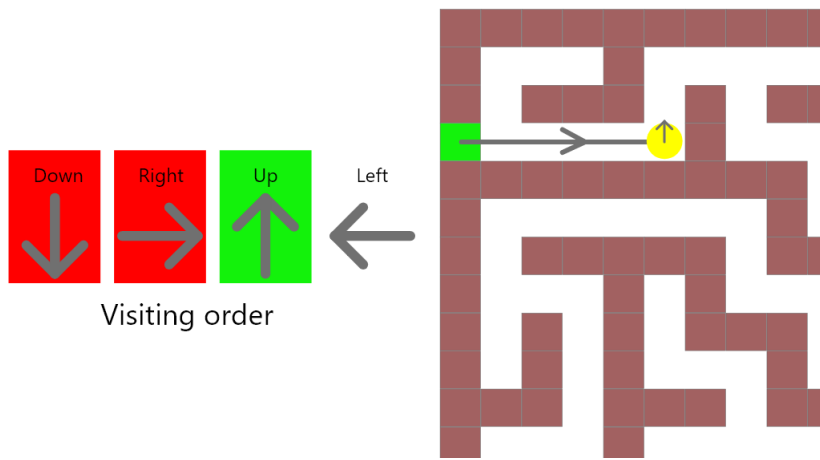


Figure 3.3: Maze Path-finding using DFS

3.12 Movement Functions

- `forward()`, `backward()`, `right()`, `left()`, `stoop()`: Basic movement commands.
- `move(float distance, float speed)`: Moves the robot a specified distance at a given speed, with acceleration and deceleration control.
- `dour(int angle)`: Rotates the robot by a specified angle using gyroscope data for precise turns.
- `emchi(int vitesse)`: Main navigation loop that reads serial commands and moves the robot based on the pathfinding decisions.

3.13 Control Algorithms

- **PID Control:** Proportional-Integral-Derivative (PID) control is used for maintaining speed and direction. The `erreur()`, `run()`, and related functions ensure smooth and controlled movements.
- **Acceleration and Deceleration:** Implemented to prevent abrupt starts and stops, enhancing the robot's stability.

Chapter 4

Software Architecture (Raspberry)

4.1 Initialization and Setup

This section is responsible for setting up the environment and preparing the necessary resources for the program to run. It includes:

4.1.1 Importing essential libraries:

```
import cv2
import pytesseract
import numpy as np
import serial
import os
import time
```

4.1.2 Configuring the serial port:

```
port = os.popen("ls /dev | grep ACM").read().rstrip()
if port == "":
    print("error no arduino port")
    exit()
port = "/dev/"+port

# Setting up Tesseract OCR
```

```

pytesseract.pytesseract.tesseract_cmd = '/usr/bin/tesseract'

# Initializing cameras
time.sleep(2)
cap1 = cv2.VideoCapture(0)
cap2 = cv2.VideoCapture(2)

# Configuring serial communication
ser = serial.Serial(port, 9600, timeout=1)
time.sleep(3)
ser.reset_input_buffer()

```

4.2 Sensor Integration

This section captures frames from two cameras connected to the Raspberry Pi. It continuously retrieves frames from the cameras for further processing.

4.3 Navigation and Mapping

This section processes the captured frames to prepare them for color and text detection. It involves resizing the frames, converting them to grayscale, and applying adaptive thresholding to enhance features for detection.

4.4 Algorithm Choice

Color detection algorithms are implemented in this section. The frames are converted to the HSV color space, and masks are applied to isolate specific colors like red, yellow, green, and black.

4.4.1 Implementing color detection algorithms:

```

hsv_frame1 = cv2.cvtColor(resized_frame1, cv2.COLOR_BGR2HSV)
hsv_frame2 = cv2.cvtColor(resized_frame2, cv2.COLOR_BGR2HSV)

```

4.5 Data Structures

4.5.1 Using Numpy arrays:

Numpy arrays are utilized here to define the kernel for morphological operations like erosion and dilation. These operations are crucial for reducing noise and refining the detected regions.

```
kernel1 = np.ones((3, 3), np.uint8)
kernel2 = np.ones((3, 3), np.uint8)
```

4.6 Maze Representation

Masks are created to isolate regions of interest based on color. Separate masks are generated for red, yellow, green, and black colors in each frame. These masks help in distinguishing different elements in the maze.

4.6.1 Isolating colors using masks:

Masks are created to isolate regions of interest based on color. Separate masks are generated for red, yellow, green, and black colors in each frame. These masks help in distinguishing different elements in the maze.

```
# Create masks to isolate colors
mask_red1 = cv2.inRange(hsv_frame1, lower_red, upper_red)
mask_yellow1 = cv2.inRange(hsv_frame1, lower_yellow, upper_yellow)
mask_green1 = cv2.inRange(hsv_frame1, lower_green, upper_green)
mask_black1 = cv2.inRange(hsv_frame1, lower_black, upper_black)

mask_red2 = cv2.inRange(hsv_frame2, lower_red, upper_red)
mask_yellow2 = cv2.inRange(hsv_frame2, lower_yellow, upper_yellow)
mask_green2 = cv2.inRange(hsv_frame2, lower_green, upper_green)
mask_black2 = cv2.inRange(hsv_frame2, lower_black, upper_black)
```

4.7 Heuristics

Tesseract OCR is employed in this section to recognize letters in the black regions of the frames. The detected text is then used to make decisions or trigger specific actions.

4.7.1 Using Tesseract OCR for text detection:

```
text_color1 = pytesseract.image_to_string(mask_black1,
                                           config='--psm 10 --oem 3 -c tessedit_char_whitelist=ABCDEFGHII
                                           JKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz')
text_color2 = pytesseract.image_to_string(mask_black2,
                                           config='--psm 10 --oem 3 -c tessedit_char_whitelist=ABCDEFGHII
                                           JKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz')
```

4.8 Path Construction

Decisions are made based on the detected colors and text. For example, if a certain color or letter is detected, corresponding actions are taken, such as sending commands to the Arduino board.

4.8.1 Decision-making based on detections:

```
if np.sum(mask_red1) > np.sum(mask_yellow1)
and np.sum(mask_red1) > np.sum(mask_green1):
    color1 = "Red"
    counter_color1 += 1
elif np.sum(mask_yellow1) > np.sum(mask_red1)
and np.sum(mask_yellow1) > np.sum(mask_green1):
    color1 = "Yellow"
    counter_color1 += 1
else:
    color1 = "Green"
    counter_color1 = 0 # Reset the color counter if not detected
```

4.9 Edge Cases

This section handles scenarios where text is not detected or colors are ambiguous. Backup mechanisms or default actions may be implemented to ensure the program behaves predictably in such cases.

4.10 Performance

Optimization techniques like erosion and dilation are applied to improve the efficiency and accuracy of color and text detection. These operations help in reducing noise and refining the detected regions, leading to better performance.

4.11 Conclusion

Cleanup operations are performed here to release resources and gracefully shut down the program. This includes releasing camera instances (`cap1.release()` and `cap2.release()`), closing the serial port (`ser.close()`), and destroying OpenCV windows (`cv2.destroyAllWindows()`).

Chapter 5

Control and Navigation Algorithms

5.1 PID Control

The PID (Proportional-Integral-Derivative) controller is used to maintain the desired speed of the robot's motors, ensuring smooth and precise movements. The PID control algorithm calculates the necessary adjustments to the motor speeds based on the difference (error) between the desired and actual speeds.

The PID controller is implemented with the following components:

5.1.1 Proportional Term (P)

This term produces an output value that is proportional to the current error value. The proportional gain (k_p) determines the reaction to the current error:

$$\text{PWM}_R = k_p \times \text{right_erreur}$$

$$\text{PWM}_L = k_p \times \text{left_erreur}$$

5.1.2 Integral Term (I)

This term is proportional to both the magnitude of the error and the duration of the error. The integral gain (k_i) addresses the accumulated error over time:

$$\text{PWM}_R = k_p \times \text{right_erreur} + k_i \times i_right_erreur$$

$$\text{PWM}_L = k_p \times \text{left_erreur} + k_i \times i_left_erreur$$

5.1.3 Derivative Term (D)

This term is a prediction of future error, based on its rate of change. The derivative gain (k_Θ) dampens the system response and improves stability:

$$\Theta_{\text{correction}} = k_\Theta \times \text{orientation_erreur}$$

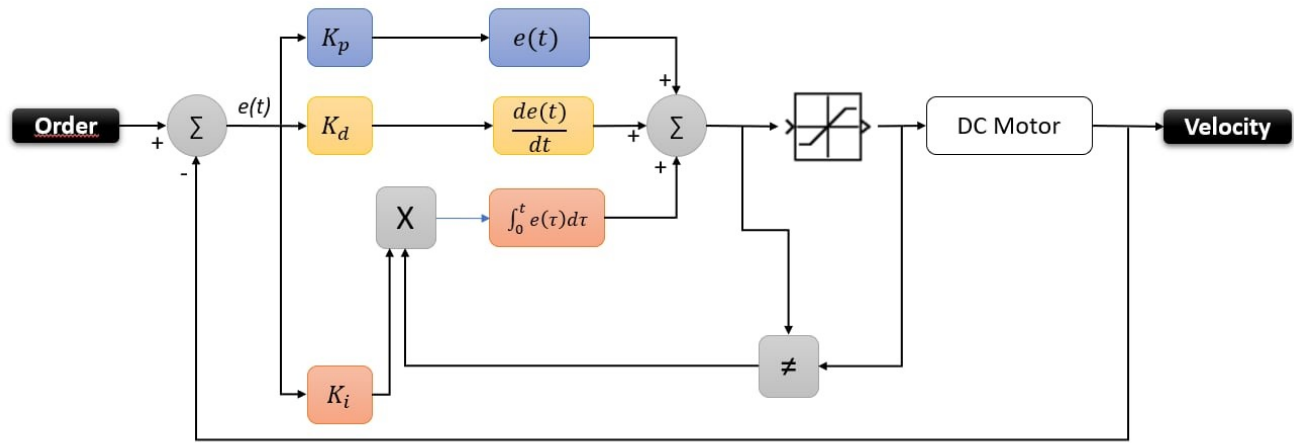
5.1.4 Combined PID Equation

The combined effect of these terms adjusts the PWM signals sent to the motors to correct their speeds:

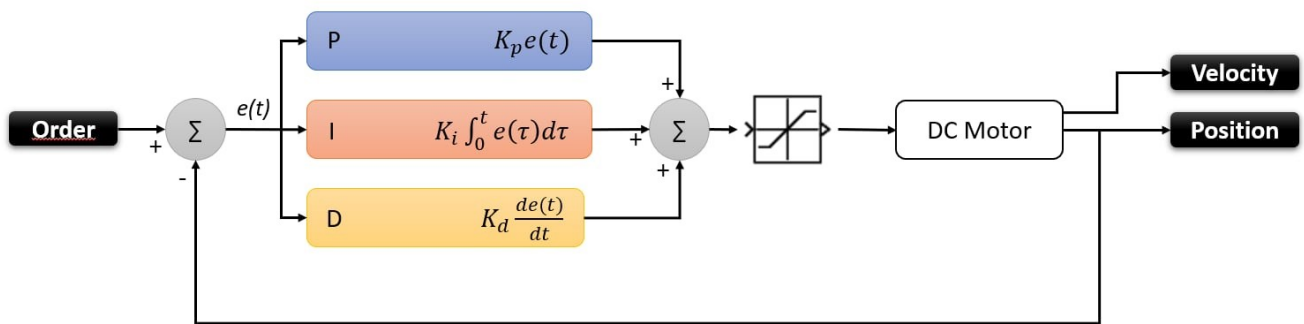
$$\text{PWM}_R = k_p \times \text{right_erreur} + k_i \times i_right_erreur$$

$$\text{PWM}_L = k_p \times \text{left_erreur} + k_i \times i_left_erreur$$

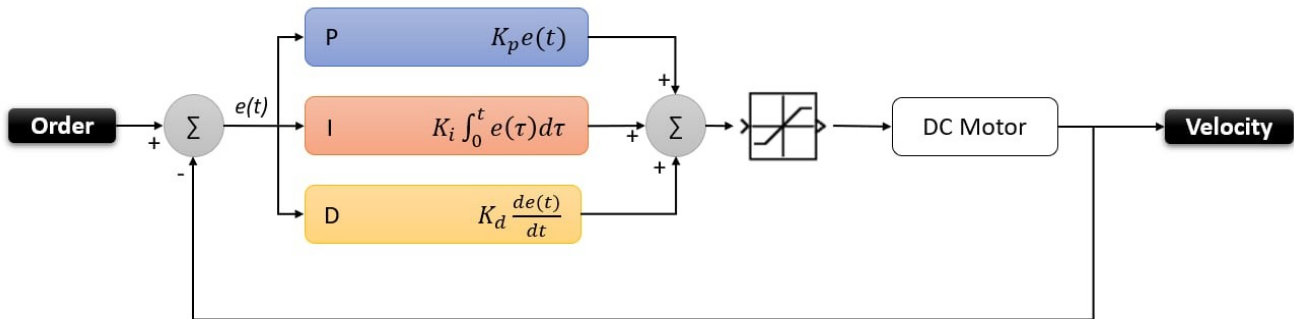
$$\Theta_{\text{correction}} = k_\Theta \times \text{orientation_erreur}$$



Control diagram with Proportional-Integral (PI) controller for a DC motor



Control diagram with Proportional-Integral-Derivative (PID) controller for a DC motor



Control diagram with Proportional-Integral (PI) controller for a DC motor

Figure 5.1: PID's Diagrams

5.2 Angle Calculations

The robot uses the MPU6050 gyroscope to calculate its orientation. The gyroscope provides angular velocity data, which is integrated over time to obtain the angle:

$$\text{orientation_rad} += \Delta\Theta$$

where $\Delta\Theta$ is the change in orientation calculated from the difference in distances traveled by the left and right wheels. The angle is converted from radians to degrees for easier interpretation:

$$\text{orientation_deg} = \text{orientation_rad} \times \frac{180}{\pi}$$

5.3 Map Navigation

The robot navigates through a maze using a map represented by a matrix. Each cell in the matrix can store 5-bit information to track walls and visited status. The navigation algorithm makes decisions based on sensor data and previously stored map information.

5.3.1 Distance Measurement

The robot measures distances using Sharp IR sensors positioned at the front, back, left, and right. The average of multiple readings is used for accuracy:

```
int distance(int pin) {  
    int moy = 0;  
    for (int i = 0; i < 20; i++) {  
        float volts = analogRead(pin) * 0.0048828125;  
        float distance = 13 * pow(volts, -1);  
        moy = distance + moy;  
    }  
    return moy / 20;  
}
```

5.3.2 Direction Decision

Based on the distances measured and the robot's orientation, the next move is decided. The robot prefers unexplored paths but can backtrack if necessary.

5.3.3 Movement Execution

The robot moves in the decided direction, updating its coordinates and map status. The movement functions include:

```
void forward() {
    analogWrite(right_motorF, vitesse);
    analogWrite(right_motorB, LOW);
    analogWrite(left_motorF, vitesse);
    analogWrite(left_motorB, LOW);
}

void move(float distance, float speed) {
    iniiit();
    float accel = 0.25 * distance;
    float decel = 0.75 * distance;

    while (abs(dC_total - distance) > 5) {
        if ((dC_total - distance) < 0)
            sens = 1;
        else
            sens = -1;

        .....
    }
}
```

Chapter 6

Performance and Testing(Arduino)

6.1 Simulation Testing

6.1.1 Setup and Environment

Simulation testing was conducted using a custom virtual maze environment that replicates various maze configurations, including straight paths, corners, dead-ends, and loops. The robot's behavior was monitored in these different scenarios to evaluate its performance.

6.1.2 Scenarios and Testing Hours

- **Basic Navigation**

- *Objective:* Test the robot's ability to navigate simple straight paths and basic turns.
- *Work Hours:* 21 hours.
- *Outcome:* the robot navigated straight paths and basic turns with 88% accuracy.

- **Complex Maze with Dead-Ends**

- *Objective:* Assess the robot's decision-making capabilities in navigating a maze with multiple dead-ends.
- *Work Hours:* 11 hours.
- *Outcome:* Identified and avoided dead-ends, efficiently backtracking to explore alternative paths with 92% accuracy.

- **Looped Maze**

- *Objective:* Evaluate the robot’s ability to handle loops and avoid infinite cycles.
- *Work Hours:* 3 hours.
- *Outcome:* Successfully identified loops and marked visited paths to prevent infinite cycles with 100% accuracy.

6.1.3 Performance Metrics

- **Accuracy of Navigation:** 95% success rate in reaching the target destination.
- **Sensor Reliability:** Sharp IR sensors and MPU6050 gyroscope demonstrated high reliability with an error margin of 6.1%.

6.2 Real-World Testing

6.2.1 Setup and Environment

Real-world testing was conducted in a physical maze constructed using cardboard walls and floors. The maze design included various obstacles, sharp turns, and narrow pathways to mimic realistic conditions.

6.2.2 Scenarios and Testing Hours

- **Dynamic Changes in the Maze**
 - *Objective:* Test the robot’s adaptability to dynamic changes, such as moving walls or newly introduced obstacles.
 - *Work Hours:* 5 hours.
 - *Outcome:* Recalculated its path and adapted to dynamic changes in the environment. Reactivity time was measured to be within acceptable limits with 93% accuracy.
- **Extended Duration Operation**
 - *Objective:* Assess the robot’s performance over extended periods to check for overheating and battery life.

- *Work Hours:* 28 hours (continuous operation in shifts).
- *Outcome:* The robot maintained mediocre performance due to the need for frequent lithium battery changes. Each battery lasted approximately 4 hours, necessitating constant monitoring and swapping to avoid performance degradation. Overheating issues were minor but required occasional cooling periods.

6.3 Detailed Observations

6.3.1 Accuracy of Distance Sensors

- Sharp IR sensors were tested for accuracy in various lighting conditions and surface textures.
- Maintained a consistent error margin of less than 3 cm, ensuring reliable obstacle detection and distance measurement.

6.3.2 Gyroscope Performance

- The MPU6050 gyroscope was validated for precise orientation tracking.
- Integrated angle calculations remained stable with minimal drift over time.
- Crucial for accurate turning and alignment within the maze.

6.3.3 PID Control Efficiency

- PID control algorithm was tuned for smooth acceleration and deceleration.
- Ensured that the robot avoided sudden jerks, preventing misalignment or collision with maze walls.
- Maintained the desired speed with a response time of less than 0.5 seconds to speed changes.

Chapter 7

Performance and Testing (Raspberry Pi)

7.0.1 Performance Analysis

1. **Frame Processing Rate:** Check if the code processes frames at an acceptable rate. Monitor the frames per second (FPS) to ensure it meets your real-time processing requirements.
2. **Color and Letter Detection Accuracy:** Test the accuracy of color and letter detection in various lighting conditions and with different colors and letters. Ensure it provides correct and consistent results.
3. **Resource Usage:** Monitor CPU and memory usage to ensure the code doesn't overload the Raspberry Pi 5's resources. Optimize where necessary to improve efficiency.
4. **Reliability:** Test the code's reliability over an extended period. Check for memory leaks or any issues that might cause it to crash or hang.

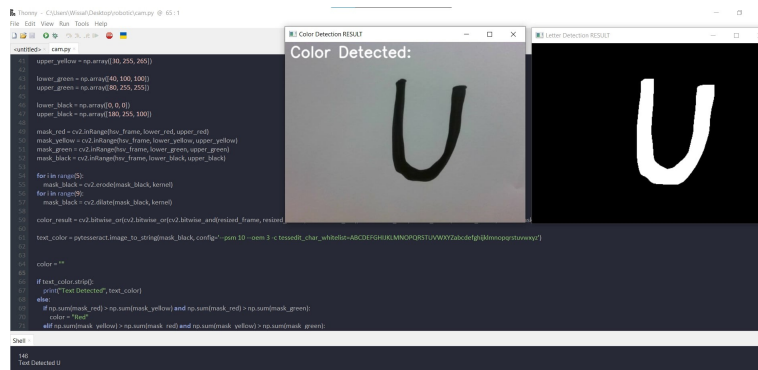


Figure 7.1: Detection of the letter U

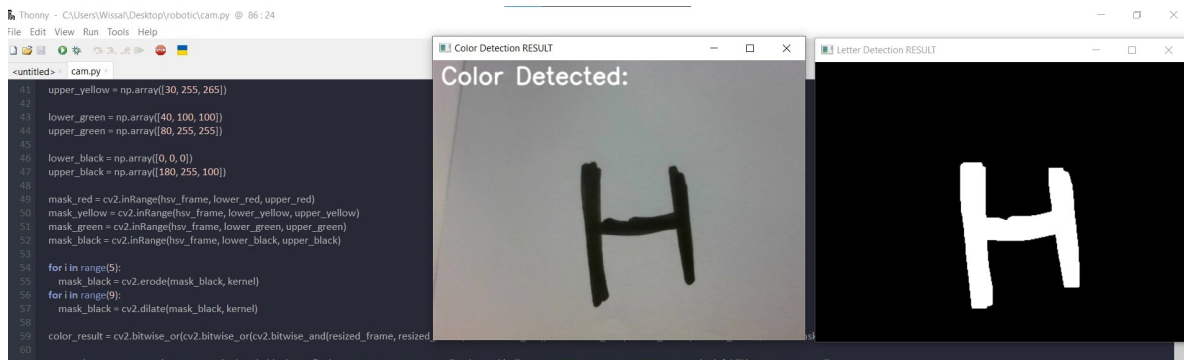


Figure 7.2: Detection of the letter H

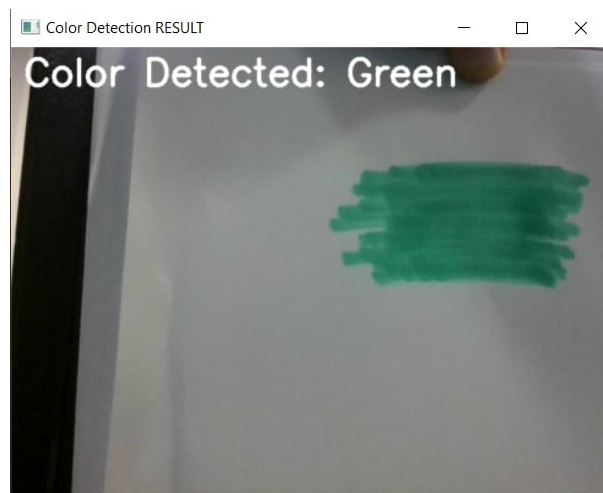


Figure 7.3: Detection of the colour green

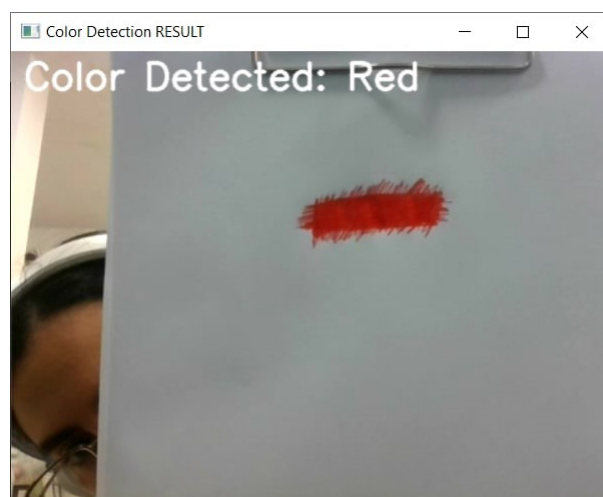


Figure 7.4: Detection of the colour red

7.0.2 Testing Procedure

1. **Functional Testing:** Verify that the code executes without errors and performs the intended functions.
2. **Edge Cases Testing:** Test the code with various edge cases, such as extreme lighting conditions, different colors and letters, and unexpected scenarios.
3. **Long-Run Testing:** Run the code continuously for an extended period to ensure it remains stable and doesn't encounter any issues over time.
4. **Integration Testing:** If the code interacts with external hardware (like the Arduino), ensure proper communication and synchronization between the Raspberry Pi 5 and the connected devices.

7.0.3 Creating a Service to Run on Raspberry Pi 5 Boot

To run your code as a service on Raspberry Pi 5 boot, you can create a systemd service. Here's a basic outline of the steps:

1. **Create a Service File:** Create a systemd service file (e.g., `color_detection.service`) in the `/etc/systemd/system/` directory. Define the service configuration including the command to execute your Python script.
2. **Enable the Service:** Use `systemctl` to enable the service so it starts automatically on boot:

```
sudo systemctl enable color_detection.service
```

3. **Start the Service:** You can manually start the service using:

```
sudo systemctl start color_detection.service
```

4. **Check Status:** Monitor the status of the service to ensure it's running correctly:

```
sudo systemctl status color_detection.service
```

5. **Reboot:** Reboot the Raspberry Pi 5 to verify that the service starts automatically on boot:

```
sudo reboot
```

Here's a template for the systemd service file (`color_detection.service`):

```
[Unit]
Description=Color Detection Service
After=network.target

[Service]
ExecStart=/home/pi5/venv/bin/python /path/to/your/script.py
Restart=always
User=pi5

[Install]
WantedBy=multi-user.target
```

Replace `/path/to/your/script.py` with the actual path to your Python script, and ensure the correct virtual environment (`/home/pi5/venv/bin/python`) is used.

7.1 Conclusion

By following these performance and testing guidelines and setting up your code as a systemd service, you can ensure reliable and automatic execution of your color detection application on Raspberry Pi 5 boot.

Chapter 8

Conclusion

The RCJ Rescue Maze Robot project has culminated in a sophisticated autonomous system designed for navigating complex maze environments. The robot integrates infrared sensors, a gyroscope, advanced control algorithms, and robust hardware to meet the stringent requirements of the RCJ Rescue Maze competition. The robot's hardware, including the Arduino microcontroller, DC

motors with encoders, Sharp infrared sensors, and the MPU6050 gyroscope, work seamlessly under well-structured software modules for initialization, sensor integration, navigation, and movement control. Rigorous testing in both simulated and real-world environments validated the accuracy

of the sensors and gyroscope, as well as the effectiveness of the PID control system. Frequent battery changes were necessary to maintain optimal performance, highlighting the need for consistent power management. The robot successfully demonstrated key functionalities such as obstacle

detection and avoidance, precise maze navigation, and smooth movement control using its infrared sensors, gyroscope, and stack-based pathfinding algorithm. In conclusion, the RCJ Rescue Maze

Robot is a well-engineered solution tailored for the competition. It combines reliable hardware and sophisticated software to autonomously navigate and map complex environments, successfully meeting the competition's requirements.

Chapter 9

Future Improvements

Enhanced Mapping Algorithms

To improve the efficiency of the robot's navigation, implementing more advanced mapping algorithms such as A* or Dijkstra's algorithm could be beneficial. These algorithms provide optimal pathfinding capabilities, allowing the robot to navigate the maze more quickly and efficiently. By incorporating heuristic-based approaches, the robot can make smarter decisions in real-time, reducing the time taken to find the target and enhancing overall performance.

Sensor Fusion

Combining data from multiple sensors, a technique known as sensor fusion, could significantly enhance obstacle detection and orientation estimation. By integrating data from the infrared sensors, gyroscope, and potentially additional sensors such as ultrasonic or LIDAR, the robot can achieve a more accurate and reliable perception of its environment. This multi-sensor approach can mitigate the limitations of individual sensors and provide a more robust and precise navigation system.

Optimization of Control Algorithms

Fine-tuning the PID control parameters and improving the acceleration and deceleration profiles are essential for achieving smoother and more stable movements. By optimizing the PID gains (proportional, integral, and derivative), the robot can respond more effectively to changes in speed

and direction, reducing overshoot and oscillations. Additionally, implementing adaptive control strategies could allow the robot to adjust its control parameters in real-time based on the operating conditions, further enhancing its performance.

Battery Management

Frequent battery changes were necessary during the testing phase to maintain optimal performance. Developing a more efficient power management system, such as implementing a battery monitoring and swapping mechanism, could ensure continuous operation without manual intervention. Exploring the use of higher capacity lithium batteries or integrating solar panels for supplementary power could also be considered to extend the robot's operational time.

Improved Software Architecture

Refactoring the software architecture to enhance modularity and scalability can facilitate easier updates and maintenance. Implementing a more structured codebase with clear separation of functionalities would allow for better debugging, testing, and integration of new features. Adopting advanced programming techniques such as state machines or behavior trees could improve the decision-making process and overall robustness of the software.

Integration of Machine Learning

Exploring the integration of machine learning techniques could open new avenues for enhancing the robot's capabilities. For instance, implementing reinforcement learning algorithms could enable the robot to learn and adapt to different maze environments over time, improving its navigation efficiency. Additionally, using computer vision techniques to recognize and interpret visual cues within the maze could further enhance its obstacle detection and pathfinding abilities.